

StructIR: Source-Adaptive Table Retrieval for LLM Data Agents

Anonymous Author(s)^{1*}

^{1*} Anonymous Institution.

Corresponding author(s). E-mail(s): anonymous@example.com;

Abstract

LLM-powered data agents increasingly operate over organizational data lakes that contain thousands of heterogeneous tables. A core step in such workflows is table retrieval: before an agent can reason over data, it must first locate the relevant table evidence. Existing retrieval tools are typically built as fixed pipelines, applying the same serialization format, retrieval signals, fusion rule, and reranking stage to every corpus. We show that this fixed-pipeline assumption breaks in heterogeneous table data lakes: compact Wikipedia-style tables benefit from dense retrieval and reranking, while long hierarchical financial tables favor sparse and fused signals and can be degraded by fixed cascades.

We propose **StructIR**, a corpus-profile-driven table retrieval skill for data agents. StructIR follows an “offline fit, online retrieve” recipe: fit computes observable corpus statistics such as link coverage, header hierarchy, lexical specificity, and reranker-window fit, then freezes a retrieval plan that selects the registered signals, fusion weights, serialization mode, and output stage. The fitted skill exposes a stable `fit/retrieve/as_tool` interface, making StructIR pluggable into different agents, readers, and retrieval backends without constraining the agent’s planning policy.

Across eight IBM Table-Text IR datasets, StructIR raises the average score over the strongest existing baseline from 73.3 to 74.9 (+2.2% relative). The gain is especially clear on financial tables: StructIR improves over the fixed cascaded retriever CRAFT by 40.1% on MultiHierTT and 73.6% on AIT-QA. Pipeline decomposition shows that these gains come from avoiding non-monotonic retrieval stages: reranking boosts compact corpora but hurts long hierarchical tables. Under a fixed 35B reader, StructIR’s retrieved evidence closes 72.7% of the BM25-to-gold-table QA gap on NQ-Tables. Finally, on a 190K-table mixed lake spanning two source families and queried end-to-end by a tool-calling agent, every fixed retrieval tool fails on some source family, while StructIR’s self-routed retrieval beats the strongest fixed pipeline by close to four points of micro R@5 with 99% routing accuracy from a label-free fit-time self-diagnostic; under an identical agent scaffold, swapping a fixed dense tool for StructIR improves every

corpus family, and a lightweight agent contract (verbatim-first querying, session-evidence union) keeps autonomous tool calls monotonically helpful across two very different LLMs.

Keywords: Table retrieval, Data lakes, LLM data agents, Reciprocal rank fusion, Source-adaptive retrieval

Large language model (LLM) data agents are becoming a practical interface for data-driven decision making over enterprise data assets Luo et al. (2026). In such settings, the agent operates over data lakes that aggregate structured and semi-structured tables from multiple sources, including financial reports, knowledge bases, statistical yearbooks, and technical documentation Wang et al. (2025); Fan et al. (2023). Before the agent can answer an analytical question, generate SQL, or inspect evidence, it must first locate the relevant tables from the available corpus. This makes *table retrieval*—returning a small ranked set of relevant tables for a natural-language query—a basic data-access skill for LLM agents over table data lakes. Recent studies on LLM interfaces for databases show that large schemas and long table contexts already strain current context windows, and irrelevant tables can degrade subsequent schema linking and reasoning quality Biswal et al. (2025); Chung et al. (2025).

Prior table retrieval methods have mainly focused on how queries and tables should be represented and matched. Early work ranks tables using semantic features over table fields, entities, and metadata Zhang and Balog (2018); Shraga et al. (2020). Dense retrieval methods then linearize tables and train dual encoders for open-domain table question answering Herzig et al. (2021); Huang et al. (2022). More recent benchmarks and systems compare sparse lexical retrieval, dense table embeddings, field-aware matching, and cascaded retrieval pipelines Ji et al. (2024); Li et al. (2025); Singh et al. (2025). These methods have substantially improved table retrieval quality. Yet most of them apply a fixed retrieval configuration uniformly across all corpora in the data lake: the same serialization, signal mix, and fusion pipeline. *The same packaging carries over to the emerging generation of data agents and table-discovery systems Cui et al. (2025); Li and Galhotra (2026); An et al. (2025); Salemi et al. (2025): the agent decides when to retrieve and what to ask, but never how retrieval is configured for the corpus at hand.*

This design becomes fragile in heterogeneous table data lakes, where tables differ in structure, metadata, and linkage patterns Eltabakh et al. (2023); Cong et al. (2023). For example, webtables Cafarella et al. (2008) often contain entity-rich cells and hyperlinks that define useful table–entity or table–passage structure; *financial* tables Chen et al. (2021b) contain sparse domain terms, nested headers, and long hierarchical layouts. *Software and technical tables* Hulsebos et al. (2023) often rely on metadata, units, sections, and compact indicator names. These structural differences directly affect retrieval behavior: metadata and field choices can change dense retrieval performance, and flat linearization can lose header hierarchy and cell–column associations that are important for table understanding Gomm and Hulsebos (2025); Wang

et al. (2026). A retriever that treats all corpora as flat text has limited ability to identify when exact lexical matching should dominate, when dense semantic matching is reliable, when a graph signal is available, or when reranking is worth its token budget.

Example. Two real queries from [table IR benchmarks](#) illustrate this issue. In NQ-TABLES, the query “when was AT&T stadium built in San Francisco” targets the AT&T Park table. BM25 misses the gold table beyond the top 100, while the dense encoder ranks it first; a uniform BM25–Dense fusion then drops the table to rank 23 by mixing in the failed sparse signal. In AIT-QA, the query “2018 net income in the March quarter for United Airlines” exhibits the opposite pattern: BM25 ranks the gold table first, the dense encoder misses it beyond the top 100, and the same uniform fusion drops the table to rank 8. Thus a fixed fusion policy can suppress the dominant signal in both directions. [Nor can the agent rescue a fixed tool at query time: in a live tool-calling loop, free query reformulation by a strong LLM degrades retrieval \(hit@5 84.6 for verbatim questions against 48.9 after rewriting\), and agent-side source selection fails exactly on same-family corpora whose contents overlap \(Section 5.5\). The adaptation has to live inside the tool, decided before queries arrive.](#) StructIR addresses this by fitting a [corpus-profile-driven](#) retrieval plan before online queries arrive.

[Section 2 details this analysis.](#) Across eight IBM Table-Text IR datasets [Awasthy et al. \(2025\)](#); [IBM Research \(2025\)](#) spanning web-table, financial, statistical, and technical-document sources, the strongest retrieval signal changes with the corpus. Dense retrieval is highly effective on several Wikipedia-style datasets; sparse matching remains strong on financial and documentation corpora; naive equal-weight fusion can dilute the best signal. Link-derived graph signals are useful when the corpus exposes cell-level links, and reranking helps when the selected serialization fits the cross-encoder budget. These findings suggest that table retrieval over heterogeneous data lakes should be configured at the [corpus-profile level](#) rather than fixed globally. As data lakes grow and new sources are onboarded continuously, a retrieval skill that automates this configuration at fit time is more sustainable than per-source manual tuning.

We propose **StructIR**, a [corpus-profile-driven](#) table retrieval skill for LLM data agents. StructIR follows an “offline fit, online retrieve” lifecycle. In the fit phase, StructIR profiles the target corpus by computing observable structural features such as cell-link coverage and header nesting depth, registers the retrieval signals supported by its profile, sets [corpus-adaptive](#) fusion weights, selects a serialization mode and output stage, and builds the corresponding indexes. In the retrieve phase, StructIR executes the frozen retrieval plan for each query and returns ranked tables through the same interface across corpora. The `as_tool` interface exposes the configured skill to an agent or downstream application. [At the lake level, fit additionally synthesizes a routing policy: it measures, without labels, when its own routing probe cannot distinguish sibling corpora and switches those source families to comparable-scale merging; and a lightweight agent contract \(verbatim-first querying, session-evidence union\) makes the agent’s autonomous tool calls monotonically helpful instead of destructive.](#) As a data lake evolves, a new source can be onboarded by running fit on the new corpus; the agent-facing retrieval API remains unchanged.

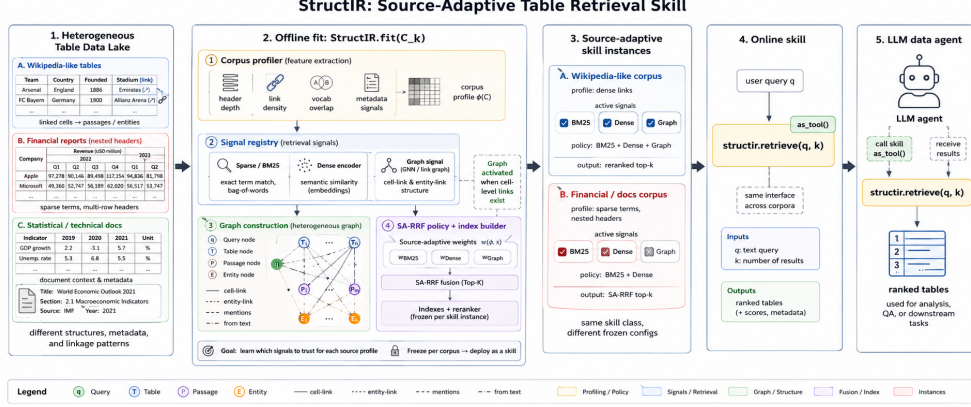


Fig. 1: Overview of StructIR. Offline profiling configures source-adaptive retrieval signals, while online calls use a unified retrieval-skill interface.

This paper makes three contributions.

- We present a cross-source analysis of table retrieval signals over eight IBM Table-Text IR datasets, showing that sparse, dense, graph-based, and reranking signals have **corpus-profile-dependent** effectiveness.
- We design StructIR, a **corpus-profile-driven retrieval skill** that turns corpus profiling into a frozen retrieval plan: signal registration, **corpus-adaptive SA-RRF fusion**, serialization selection, and output-stage selection.
- We evaluate StructIR on the full IBM Table-Text IR suite, downstream QA settings, and a **live mixed-lake agent benchmark** (190K tables, four sources, two source families) queried end-to-end by a tool-calling agent, showing that a **profile-driven retrieval skill** improves table retrieval quality, transfers to higher end-task accuracy under a fixed reader, and **beats every fixed retrieval tool given to the same agent infrastructure**: self-routed retrieval outperforms the strongest fixed pipeline by close to four points of micro R@5 with 99% routing accuracy, and swapping a fixed dense tool for StructIR under an identical agent adds more than five points.

1 Problem Definition

1.1 Heterogeneous Table Data Lake

A *table data lake* is a repository that aggregates table collections from multiple sources Fan et al. (2023). We model a heterogeneous table data lake as $\mathcal{L} = \{D_1, \dots, D_K\}$, where each *source* D_k contributes a table corpus $\mathcal{C}_k = \{T_1, \dots, T_{n_k}\}$. Each table T_i consists of column headers, cell values, and optionally source-dependent metadata (e.g., page titles in Wikipedia, filing identifiers in financial reports). Sources in $\mathcal{L}$ may differ in schema conventions, vocabulary distributions, header complexity, and the availability of structural metadata such as cell-level entity hyperlinks Eltabakh et al. (2023); Cong et al. (2023). These differences, which we quantify through *corpus*

profiles in Section 2, are the root cause of the retrieval challenge addressed in this paper.

1.2 Table Retrieval Task

Following Zhang and Balog [Zhang and Balog \(2018\)](#) and Ji et al. [Ji et al. \(2024\)](#), we define the table retrieval task as: given a natural language query q and a table corpus $\mathcal{C}_k$, return a ranked list of tables from $\mathcal{C}_k$ ordered by relevance to q . When the data lake $\mathcal{L}$ contains multiple sources, the retrieval system may be instantiated once per source, each operating over $\mathcal{C}_k$ independently. We evaluate retrieval quality using Recall@ K [Ji et al. \(2024\)](#) and, for datasets with decontextualized queries, Match@ K [Awasthy et al. \(2025\)](#).

1.3 Retrieval Skill for Data Agents

Recent data-agent frameworks characterize agents as systems that interact with large, heterogeneous data assets through specialized tools across the data lifecycle [Luo et al. \(2026\)](#). In this paper, we use this perspective to define the interface between an agent and a table retriever. StructIR is a retrieval skill that can be configured for a target corpus and then invoked through a stable tool interface.

We model a retrieval skill $\mathcal{S}$ as a stateful callable with three interfaces:

- `fit( $\mathcal{C}_k$ )`: analyze the target corpus and configure the retrieval plan. All adaptation to the corpus profile of $\mathcal{C}_k$ occurs at this stage.
- `retrieve( $q, K$ )`: given a query q , return the top- K tables from $\mathcal{C}_k$ using the plan configured at fit time. The call executes a frozen plan and does not reconfigure signals, weights, or stages at query time.
- `as_tool()`: export a function specification (name, description, parameter schema) that a data agent or downstream application can register and invoke.

This three-interface abstraction separates *corpus-level adaptation* (at fit time) from *query-time execution* (at retrieve time), and from *agent integration* (at `as_tool` time). The design goal of StructIR, detailed in Section 3, is to make the fit phase corpus-adaptive: given any new corpus $\mathcal{C}_k$ in the data lake, StructIR configures retrieval signals, fusion weights, serialization mode, and output stage from the corpus profile, so that the agent-facing retrieval API remains unchanged as $\mathcal{L}$ evolves.

2 Motivation: Cross-Source Signal Analysis

We analyze retrieval signal effectiveness across four representative corpora with distinct structural profiles from the IBM Table-Text IR evaluation suite: NQTABLES (Wikipedia, 170K tables), MULTIHIERTT (financial, 9.9K tables), WATSONXDOCS (technical documentation, 1.1K tables), and STATCAN (statistical, 5.9K tables). The goal is to identify which corpus characteristics change the relative strength of sparse, dense, graph-based, and reranking signals. We report Recall@1 throughout this section.

Table 1: BM25 vs. dense retrieval (R@1) across four representative corpora.

Method	NQTables	MultiHierTT	WatsonxDocs	StatCan
BM25	17.81	45.55	53.33	0.63
Dense	45.27	34.45	43.33	8.88

Table 2: GNN vs. Granite-R2 on two Wikipedia datasets (R@1, 5-seed mean $\pm$ std).

Method	Params	NQTables	OTT-QA
BM25	—	17.81	62.92
Granite-R2	149M	42.46	83.83
GNN (ours)	2M	44.70 $\pm$ 1.8	85.14 $\pm$ 1.2

2.1 Observation 1: Text Signal Effectiveness Depends on Corpus Profile

Table 1 compares BM25 and dense retrieval (bge-m3) across the four corpora. On NQTABLES, dense retrieval outperforms BM25 by a wide margin (45.27 vs. 17.81), consistent with the TARGET finding that dense methods dominate on Wikipedia-sourced corpora [Ji et al. \(2024\)](#). On MULTIHIERTT and WATSONXDOCS, the ranking reverses: BM25 achieves 45.55 and 53.33, surpassing dense retrieval by 11 and 10 points. In MULTIHIERTT, financial tables often contain multi-level headers and precise indicator terms. Long linearized headers weaken dense discrimination, while BM25 directly matches terms such as years, metrics, and monetary units.

Implication. Equal-weight fusion can be dragged down by the weaker signal on each corpus. Fusion weights should depend on corpus-profile features.

2.2 Observation 2: Link Graphs Provide a Strong Signal When Available

Wikipedia tables contain cell-level hyperlinks to entity pages, forming a cell–passage association graph. We train a lightweight HGTCnv GNN [Hu et al. \(2020\)](#) (2M parameters) on this graph and compare it with Granite-R2 [Awasthy et al. \(2025\)](#), a 149M-parameter table-specialized embedding model.

The GNN is competitive with Granite-R2 on both datasets while using two orders of magnitude fewer parameters. The result indicates that cell–passage link topology provides a retrieval signal beyond flat table text. In corpora without cell-level hyperlinks, this graph signal cannot be constructed.

Implication. The graph signal should be registered when link coverage is available and omitted otherwise.

Table 3: Reranker ablation on financial tables (M@1, dev subset).

Variant	AIT-QA ($n=70$)	MultiHierTT ($n=155$)
flat	60.00	54.19
shuffled	64.29	52.90
struct	67.14	56.13
struct-shuffled	+2.85	+3.23

2.3 Observation 3: Header Hierarchies Can Improve Reranking on Financial Tables

On MULTIHIERTT and AIT-QA, we compare a standard cross-encoder reranker with a structure-aware variant that preserves header-path information, e.g., *Revenue > Operating > FY2019*. We also include a shuffled condition that randomly permutes header paths, keeping token count comparable while removing hierarchy.

The struct-shuffled difference isolates the contribution of header hierarchy from token count. The positive gaps show that header paths carry useful reranking information when nested headers are present.

Implication. Corpora with multi-level headers should expose header-path information to the reranker.

2.4 Corpus Profile

The observations above show a common pattern: the effective retrieval configuration depends on observable properties of the target corpus. We summarize these properties using a compact corpus profile.

- **Hyperlink density** $\rho_{\text{link}} \in [0, 1]$: the fraction of cells containing entity hyperlinks. It indicates whether a cell-passage graph signal can be built.
- **Multi-level header fraction** $\rho_{\text{header}} \in [0, 1]$: the proportion of tables with nested headers (depth ≥ 2). It captures whether header-path serialization is relevant.
- **Oracle lexical alignment** $\rho_{\text{lex}}^* \in [0, 1]$: the normalized mean BM25 score between queries and their gold tables. It is used only as an analysis statistic for identifying BM25-dominant regimes.
- **Deployable lexical specificity** $\rho_{\text{idf}} \in [0, 1]$: a query-free proxy computed from the IDF distribution of non-stopword header and cell tokens. It captures corpus-specific terminology without relevance labels.

The diagnostic feature ρ_{lex}^* explains lexical alignment in controlled analysis. The observable features $(\rho_{\text{link}}, \rho_{\text{header}}, \rho_{\text{idf}})$ provide the deployable profile for configuring retrieval signals and fusion behavior.

3 Method Design

StructIR is a reusable table retrieval skill for heterogeneous table data lakes. It separates *configuration* from *execution*. The configuration phase, implemented by `fit`, profiles a target corpus and freezes a corpus-specific retrieval plan. The execution

phase, implemented by `retrieve`, applies the frozen plan to each incoming query and returns top- k tables. The exported `as_tool` interface allows an LLM data agent to call the same retrieval function across different corpora.

It is useful to state the abstraction before the components. A *retrieval plan* is a point $\pi = (\mathcal{R}_\phi, \alpha_\phi, m_\phi, o_\phi)$ in a plan space Π : the registered signal set, the fusion weights over it, the table serialization the scoring stages consume, and the output stage (whether a trained reranker rescores the fused candidates or the plan stops at fusion). The individual components are deliberately standard: BM25, a frozen encoder, rank fusion, a cross-encoder. The contribution of fit is *which point of Π each corpus receives, synthesized from label-free corpus statistics and frozen before the first query*. A conventional fixed retrieval tool is the degenerate case $|\Pi| = 1$; Section 2 shows its cost is double-digit and bidirectional, and Section 5.5 shows the agent cannot pay that cost down at query time. From the agent’s perspective the skill therefore exposes exactly four things (packaged as a toolkit in Section 4): `fit`, `retrieve`, `as_tool`, and a *configuration card* documenting every frozen decision with the statistic that triggered it, so the agent (or its developer) can audit a source’s configuration without reading code. The design principle throughout: every decision the agent provably gets wrong at query time moves into fit; every decision the agent can make reliably is left to it and documented in the tool schema.

StructIR uses three reusable model resources: a frozen text encoder M_{embed} (e.g., bge-m3 Chen et al. (2024)), a graph retrieval model M_{gnn} for link-rich corpora, and a cross-encoder reranker M_{rerank} . These components are trained or selected before deployment using labeled query–table relevance data:

$$\mathcal{D}_{\text{train}} = \{(q_i, T_i^+, \mathcal{T}_i^-)\}_{i=1}^n,$$

where T_i^+ is a relevant table from the qrels and $\mathcal{T}_i^-$ contains negatives sampled from BM25, Dense, or SA-RRF candidate lists. The fit phase then adapts these resources to a new corpus by configuring the active signals, fusion weights, serialization mode, output stage, and indexes.

3.1 Configuration Phase: Corpus-Driven Plan

Given a target table corpus $\mathcal{C}_k$ and an optional calibration split $\mathcal{V}_k$, StructIR produces a frozen retrieval plan:

$$\text{fit} : (\mathcal{C}_k, \mathcal{V}_k) \rightarrow (\phi, \mathcal{R}_\phi, \alpha_\phi, m_\phi, o_\phi, \text{Idx}).$$

Here ϕ is the corpus profile, $\mathcal{R}_\phi$ is the registered signal set, α_ϕ is the SA-RRF weight vector, m_ϕ is the serialization mode, o_ϕ is the output stage, and `Idx` contains all indexes required by the frozen plan.

Corpus profile. The profiler computes observable properties of the target corpus:

$$\phi(\mathcal{C}_k) = (\rho_{\text{link}}, \rho_{\text{header}}, \rho_{\text{idf}}, \rho_{\text{len}}) \in [0, 1]^4.$$

ρ_{link} measures the coverage of cell-level links or entity references that can support graph construction. ρ_{header} measures the depth and frequency of nested or multi-row headers. ρ_{idf} measures vocabulary specificity, computed from non-stopword header and cell tokens without using relevance labels. ρ_{en} measures the fraction of candidate table serializations that fit within the cross-encoder token budget. These statistics describe retrieval affordances of the corpus rather than a manually assigned source-family label.

Signal registry. BM25 and Dense retrieval are always registered. The graph signal is registered when the corpus contains a usable link graph:

$$\mathcal{R}_\phi = \{\text{BM25, Dense}\} \cup \begin{cases} \{M_{\text{gnn}}\} & \rho_{\text{link}} \geq \tau_{\text{link}}, \\ \emptyset & \rho_{\text{link}} < \tau_{\text{link}}. \end{cases} \quad (1)$$

This rule follows the availability of structural evidence. The GNN is activated when cell–entity or cell–passage links can be materialized, and BM25/Dense remain the fallback signals for corpora without such links.

Fusion weights. StructIR fuses only the registered signals. When a calibration split $\mathcal{V}_k$ is available, the SA-RRF weights are learned by optimizing retrieval quality on $\mathcal{V}_k$:

$$\alpha_\phi = \text{softmax}(\mathbf{W}_\pi \phi), \quad \alpha_\phi \in \Delta^{|\mathcal{R}_\phi|}. \quad (2)$$

$\mathbf{W}_\pi$ is the only learned parameter of the fusion policy and is estimated with cross-fitting over calibration qrels. When no qrels are available for the target corpus, StructIR uses a deterministic profile heuristic: the BM25 weight increases with ρ_{idf} , the graph weight increases with ρ_{link} , and the remaining mass is assigned to Dense. In both cases, the resulting weights are frozen before online retrieval.

Serialization mode. The reranker input format is also selected from the corpus profile:

$$m_\phi = \begin{cases} \text{struct} & \rho_{\text{header}} \geq \tau_{\text{header}}, \\ \text{std} & \rho_{\text{header}} < \tau_{\text{header}}. \end{cases} \quad (3)$$

The standard mode concatenates table title, headers, and cells. The structure-aware mode prefixes each cell value with its root-to-leaf header path and is therefore triggered by nested header structure.

Indexes. Finally, fit builds the indexes required by $\mathcal{R}_\phi$: a BM25 inverted index over linearized tables, a dense vector store using M_{embed} , and a table–entity or table–passage graph when M_{gnn} is registered. The frozen plan is then reused by all subsequent calls to `retrieve`.

Agent configuration note. For an integrating agent, the defaults to keep are the global thresholds and the profile heuristic for fusion weights; the cross-fit calibration of Eq. 2 is the labeled-data upgrade of that heuristic. The override worth making is to pass a calibration split whenever even a few hundred labeled queries exist: it recovers per-corpus fusion optima that the deterministic heuristic cannot express, including corpora of the same structural profile that need opposite fusion biases (Section 5.5.1).

Forcing the reranking stage on a low- ρ_{len} corpus is the one override that is never safe (Section 5.3).

3.2 Execution Phase: Multi-Signal Retrieval

Given a query q , the execution phase runs the frozen plan:

$$\text{retrieve} : (q, k) \rightarrow \text{List}[\text{Table}].$$

Each registered signal scores the corpus independently, SA-RRF combines the ranked lists, and the selected output stage returns the final top- k tables.

3.2.1 Signal Pool

Each signal captures a different query–table alignment channel:

$$s_{\text{bm25}}(q, T) = \text{BM25}(q, \text{serialize}(T)), \quad (4)$$

$$s_{\text{dense}}(q, T) = \text{Embed}_q(q)^\top \text{Embed}_d(T), \quad (5)$$

$$s_{\text{gnn}}(q, T) = \mathbf{q}^\top \hat{\mathbf{h}}_T, \quad (6)$$

where s_{gnn} is computed only when $M_{\text{gnn}} \in \mathcal{R}_\phi$. BM25 emphasizes exact indicator and entity terms, Dense captures semantic paraphrases, and the graph signal propagates relevance through linked entities or passages that are invisible to flat serialization.

3.2.2 Graph Retrieval Model

Observation 2 motivates a third signal channel: on link-rich corpora, a table’s relevance is often carried by the passages its cells link to, which is invisible to any scorer that reads the table text in isolation. The graph model takes as input the cell–passage link structure and the frozen dense embeddings, and outputs a refined table embedding $\hat{\mathbf{h}}_T$ that backs the score s_{gnn} of Eq. 6.

Cell–passage link graph. For a link-rich corpus, StructIR constructs a heterogeneous graph

$$\mathcal{G} = (\mathcal{V}_T \cup \mathcal{V}_A, \mathcal{E}),$$

where $\mathcal{V}_T$ contains table nodes and $\mathcal{V}_A$ contains anchor nodes such as linked entities or passages. An edge $(T, a) \in \mathcal{E}$ exists iff some cell $c_{i,j}$ of table T hyperlinks to anchor a ; this is exactly the structure whose coverage ρ_{link} measures, so the graph is constructible precisely when the profile registers the signal (Eq. 1).

Node initialization and propagation. Table nodes are initialized from the frozen dense encoder:

$$\mathbf{h}_T^{(0)} = W_{\text{in}} \cdot \text{Embed}_d(\text{serialize}(T)), \quad (7)$$

and anchor nodes from the mean embedding of their neighboring tables:

$$\mathbf{h}_a^{(0)} = \frac{1}{|\mathcal{N}(a)|} \sum_{T \in \mathcal{N}(a)} \text{Embed}_d(\text{serialize}(T)). \quad (8)$$

L layers of HGTCConv [Hu et al. \(2020\)](#) propagate anchor-level context into table representations, and a gated residual combines the refinement with the dense initialization:

$$\tilde{\mathbf{h}}_T = \mathbf{h}_T^{(0)} + \sigma(g) \cdot W_{\text{out}} \mathbf{h}_T^{(L)}. \quad (9)$$

The gate is initialized to $g = -5$, so $\sigma(g) \approx 0.007$ at the start of training: the model starts at the dense representation (cold-start safety) and learns how much graph information to admit. The query is projected into the same space by $\mathbf{q} = W_q \text{Embed}_q(q)$, giving the score of Eq. 6.

Training. M_{gnn} is trained on link-rich corpora with available qrels: the gold table T^+ is the positive and hard negatives T^- are sampled from top-ranked BM25 and Dense candidates, with the InfoNCE objective

$$\mathcal{L}_{\text{gnn}} = -\log \frac{\exp(s_{\text{gnn}}(q, T^+)/\tau)}{\exp(s_{\text{gnn}}(q, T^+)/\tau) + \sum_{T^- \in \mathcal{T}^-} \exp(s_{\text{gnn}}(q, T^-)/\tau)}. \quad (10)$$

The trainable parameters are $\{W_{\text{in}}, W_{\text{out}}, W_q, g, \text{HGTCConv layers}\}$; M_{embed} stays frozen. [Registration of this signal is a profile decision \(\$\rho_{\text{link}} \geq \tau_{\text{link}}\$, Eq. 1\); whether a frozen plan deploys it is an evidence decision made by the configuration study \(Section 5.3\).](#)

3.2.3 Source-Adaptive Fusion

Standard RRF assigns equal weight to all ranked lists. StructIR uses the corpus-conditioned weights from Eq. 2:

$$s_{\text{SA-RRF}}(q, T) = \sum_{c \in \mathcal{R}_\phi} \alpha_{\phi, c} \cdot \frac{1}{\kappa + \text{rank}_c(q, T)}. \quad (11)$$

Rank-space fusion avoids score-scale mismatch across BM25, Dense, and graph scores. Since α_ϕ is fixed during fit, online retrieval introduces no query-time routing or adaptive control.

3.3 Reranking and Output-Stage Selection

The cross-encoder reranker M_{rerank} scores SA-RRF top- K_{cand} candidates by applying token-level cross-attention between the query and the serialized table. It is trained on labeled query-table pairs from $\mathcal{D}_{\text{train}}$: positive pairs are drawn from qrels, and negative pairs are sampled from candidate tables retrieved by BM25, Dense, or SA-RRF.

The serialization format is determined by m_ϕ . In standard mode,

$$[\text{CLS}] \ q \ [\text{SEP}] \ T.\text{title} \oplus T.H \oplus T.\text{cells}. \quad (12)$$

In structure-aware mode,

$$[\text{CLS}] \ q \ [\text{SEP}] \ T.\text{title} \oplus \bigoplus_{c_{i,j}} \text{path}(c_{i,j}) \| v_{i,j}, \quad (13)$$

where $\text{path}(c_{i,j})$ is the root-to-leaf header chain of cell $c_{i,j}$. This preserves header hierarchy for corpora with nested table structure.

StructIR also chooses where to stop in the pipeline. When a calibration split $\mathcal{V}_k$ is available, the stage selector compares SA-RRF and reranking under the selected serialization mode:

$$\Delta_{\text{rerank}}(\mathcal{V}_k) = \text{Metric}(M_{\text{rerank}} \circ \text{SA-RRF}; \mathcal{V}_k) - \text{Metric}(\text{SA-RRF}; \mathcal{V}_k). \quad (14)$$

The output stage is then frozen as

$$o_\phi = \begin{cases} M_{\text{rerank}}(\text{SA-RRF top-}K_{\text{cand}}) & \Delta_{\text{rerank}}(\mathcal{V}_k) > \epsilon, \\ \text{SA-RRF top-}k & \Delta_{\text{rerank}}(\mathcal{V}_k) \leq \epsilon. \end{cases} \quad (15)$$

When no calibration split is available, StructIR uses a token-budget heuristic: reranking is enabled when $\rho_{\text{len}} \geq \tau_{\text{len}}$, and SA-RRF is used directly when most candidates exceed the reranker window. Thus output-stage selection is based on observed validation gain or reranker feasibility, rather than on corpus names.

4 StructIR: A Lightweight Toolkit for Adaptive and Agentic Text2Table IR

The design of Section 3 ships as a small Python toolkit. This section states its design principles, the programming abstraction for human users and for agents, and how the multi-signal design is instantiated efficiently by model reuse.

4.1 Key Design Principles

- **Configuration lives offline.** Every decision the agent provably gets wrong at query time (signal weights, serialization, output stage, within-family routing) is synthesized and frozen by fit; `retrieve` is deterministic.
- **Plans over models.** The components are standard and reusable; onboarding a corpus never retrains a model, it selects a point in the plan space from label-free corpus statistics.
- **Self-diagnosis over assumptions.** The lake layer measures, at fit time, when its own routing probe cannot be trusted, and freezes merging instead of routing for those source families.
- **Contracts over prompts.** Agent usage rules (verbatim-first querying, session-evidence union) are enforced by the harness rather than requested in prompts.
- **Auditability.** Every frozen decision is recorded on a configuration card together with the statistic that triggered it.

4.2 Programming Abstraction (for Human Users)

The core abstraction is scikit-learn-like: a skill object is fit once per corpus and then queried (Figure 2).

```

1 from structir_skill import (
2     TableRetrievalSkill, TableLakeSkill)
3
4 # one corpus: profile -> frozen plan -> indexes
5 skill = TableRetrievalSkill(reranker_ckpt=CKPT)
6 skill.fit(corpus, queries=qs,
7           calibration=cal) # optional labels
8 print(skill.card()) # audit the plan
9 hits = skill.retrieve(question, top_k=5)
10
11 # a lake of corpora: per-source plans + routing
12 lake = TableLakeSkill(reranker_ckpt=CKPT)
13 for name, corpus, qs in sources:
14     lake.add_source(name, corpus, queries=qs)
15 out = lake.retrieve(question, source="auto")
16 tool = lake.as_tool() # one agent schema

```

Fig. 2: Minimal usage of the StructIR toolkit. Onboarding a new source is one `add_source` call; the agent-facing interface never changes.

Core interface. `fit(corpus, queries, calibration)` profiles the corpus, freezes the plan of Section 3, and builds the indexes; with a labeled calibration split it upgrades the fusion weights by leakage-free two-fold cross-fitting, and without one it falls back to the deterministic profile heuristic. `retrieve(q, top_k)` executes the frozen plan. `card()` returns every frozen decision with its triggering statistic. Precomputed corpus embeddings can be passed to avoid re-encoding large corpora (Section 4.4).

StructIR exports three typed interfaces:

$$\text{fit} : (\mathcal{C}_k, \mathcal{V}_k) \rightarrow (\phi, \mathcal{R}_\phi, \alpha_\phi, m_\phi, o_\phi, \text{Idx}), \quad (16)$$

$$\text{retrieve} : (q, k) \rightarrow \text{List}[\text{Table}], \quad (17)$$

$$\text{as_tool} : () \rightarrow \text{FunctionSpec}. \quad (18)$$

`fit` establishes the applicability condition of the skill by profiling and indexing the corpus. `retrieve` executes the frozen retrieval plan and terminates by returning top- k tables. `as_tool` exports the same callable interface to an LLM data agent, while `card()` records the corpus profile, registered signals, fusion weights, serialization mode, and output stage.

4.3 Programming Abstraction (for Agents): the Lake-Level Plan and the Usage Contract

When the lake holds multiple fitted sources, one more plan dimension must be synthesized: *where* a query executes. `retrieve(q, source)` accepts a member name, a source-family name, or `auto`.

Routing that knows when it cannot route. For `auto`, a dense probe scores the query against the whole lake (one shared encoder, hence one comparable scale) and the top- K neighbors $N_K(q)$ vote at the *family* level first; let $f^*(q)$ denote the winning family with members M_{f^*} . Within the winning family, the fit-time self-diagnostic governs. For each member m with sample queries Q_m , the lake measures the vote-leak

rate

$$\text{leak}(f) = \max_{m \in M_f} \mathbb{E}_{q \in Q_m} \left[\frac{|\{d \in N_K(q) : \text{src}(d) \in M_f \setminus \{m\}\}|}{|\{d \in N_K(q) : \text{src}(d) \in M_f\}|} \right], \quad (19)$$

and freezes the routing rule

$$r(q) = \begin{cases} \arg \max_{m \in M_{f^*}} \text{vote}_m(q) & \text{leak}(f^*) < \varepsilon, \\ \text{merge}(M_{f^*}) & \text{leak}(f^*) \geq \varepsilon. \end{cases} \quad (20)$$

High leak means the probe cannot identify a query’s origin inside that family (content-overlapping siblings), so the family is merged rather than routed; low leak means member voting is reliable. The statistic is label-free and computed once at onboarding.

Comparable-scale merging. Merging is performed only on scales that are actually comparable. Members whose plans end in *rerank* share one scorer (the same cross-encoder), so their candidates merge by its score; the criterion is shared scorer identity, not the stage label. Members whose plans stop at fusion merge by rank-RRF over each member’s own fused list, weighted by the probe-vote posterior for the query: per-source weighted-RRF *masses* are not comparable across different weight vectors, and unweighted rank-RRF ties every same-rank pair, which a stable sort then resolves deterministically in favor of one source. Both failure modes are measured in Section 5.5.1. Mixed-stage merges are refused; the skill routes instead. These rules are frozen into the lake plan and appear on the card.

The agent contract. Three usage rules close the agent-side failure modes measured in Section 5.5 and are enforced by the harness: (i) *verbatim-first*: the tool schema instructs the agent to pass the user question unchanged on the first call, and to reformulate at most once, only on clearly irrelevant results, preserving entity names; (ii) *session-evidence union*: with H_c the hits and o_c the output stage of call c , the episode’s evidence is the top- k of $\bigcup_c H_c$ under

$$\text{key}(d) = \begin{cases} \max_{c:d \in H_c} s_c(d) & |\{o_c\}| = 1, \\ \max_{c:d \in H_c} 1/\text{rank}_c(d) & \text{otherwise,} \end{cases} \quad (21)$$

so extra calls can add recall but cannot displace a good first call: scores are aggregated only when every call ends in the same stage (one scorer, one scale), and rank is used across mixed stages; (iii) *a separate fixed reader* produces the answer from the final evidence. The effect of each rule is quantified in Section 5.5.

Case example: plugging into an existing agent. With an internal ADK-style text-to-SQL agent (tools are plain Python callables), the integration is three steps and no retraining: (1) onboard each source once offline via `lake.add_source(name, corpus, queries)` and log the returned card for audit; (2) register one callable, `retrieve_tables(query, source="auto", k)`, whose schema ADK derives from the signature (non-ADK stacks take `as_tool()` *verbatim*), and place the lake catalog text in the agent’s system prompt; (3) adopt the contract: the *verbatim-first* instruction ships inside the tool description, and *session-union* aggregation is a ten-line harness change.

When a new source lands, the agent-facing interface does not change: fit synthesizes and freezes its plan, the catalog gains one line, and the routing self-diagnostic re-runs.

4.4 Efficient Multi-Signal Instantiation via Model Reuse

The multi-signal design of Section 3 would be expensive if each signal carried its own encoder; the toolkit instead shares the two pretrained models across every consumer.

One frozen encoder, three consumers. The corpus embedding matrix produced once by M_{embed} at fit time backs (i) the dense scoring leg, (ii) the node initialization of the graph candidate slot, and (iii) the lake’s routing probe and its leak self-diagnostic, which operate on the concatenation of the per-source matrices. No corpus is ever encoded twice, and online queries are encoded once per call (a cached fast path serves repeated benchmark queries).

Graph slot at zero query-time graph cost. The slot’s refinement is materialized at fit time as the gated residual of Eq. 9 over the cached dense matrix, so at retrieve time the graph signal is a vector scan over a precomputed matrix: no message passing sits on the query path.

One reranker, comparable scales for free. A single cross-encoder instance is shared by every rerank-stage source in the lake. Memory stays constant in the number of sources, and, as a by-product, candidates from different sources arrive on one score scale, which is exactly what the family-merge of Section 4.3 requires.

Calibration without model calls. The two-fold cross-fit of fusion weights consumes the per-leg rankings already produced while building indexes; the weight grid search touches only rank lists, so passing calibration adds no encoder or reranker invocations to onboarding.

5 Experiments

Goal. We evaluate whether StructIR provides a robust table-retrieval skill for heterogeneous table data lakes. The experiments answer four questions:

- **RQ1 (Overall retrieval effectiveness).** Does the frozen retrieval plan produced by StructIR improve retrieval quality over sparse, dense, fixed-fusion, pre-trained embedding, and cascaded retrieval baselines?
- **RQ2 (Stage non-monotonicity).** When does reranking improve ranking, and when does it degrade the result?
- **RQ3 (Fixed-reader utility).** Under a fixed reader, does better retrieved table evidence transfer to QA accuracy?
- **RQ4 (Agent-tool integration).** Can StructIR be invoked as a live retrieval skill through a stable agent tool interface; and on a heterogeneous mixed lake queried end-to-end by a tool-calling agent, does the self-configuring skill beat every fixed retrieval tool under an identical agent scaffold?

Evaluation protocol. RQ1 evaluates full retrieval quality on all eight IBM Table-Text IR datasets. RQ2 decomposes the retrieval pipeline into BM25, Dense, SA-RRF,

Table 4: Corpus profiles and frozen StructIR plans. Profile statistics are computed at fit time and determine signals, serialization, and output stage.

Dataset	Type	#Tbl	#Q	Metric	Corpus profile $\phi(C_k)$				Frozen plan			
					ρ_{link}	ρ_{header}	ρ_{idf}	ρ_{len}	Signals R_ϕ	Ser. m_ϕ	Out. o_ϕ	Wt. α_ϕ
OTT-QA	Wiki	8.9K	2214	R@5	1.00	0.03	0.58	0.69	BM25,Dense,(Graph)	std	Rerank	Dense
NQ-Tab.	Wiki	170K	1067	R@5	0.98	0.23	0.50	0.83	BM25,Dense,(Graph)	std	Rerank	Dense
OpenWiki	Wiki	54K	6602	R@5	0.99	0.00	0.48	1.00	BM25,Dense,(Graph)	std	Rerank	Dense
FeTaQA	Wiki	10K	2003	R@5	1.00	0.26	0.45	0.66	BM25,Dense,(Graph)	std	Rerank	Dense
MultiHT	Fin.	9.9K	929	M@5	0.00	0.58	0.70	0.36	BM25,Dense	struct	SA-RRF	BM25
AIT-QA	Fin.	1.9K	515	M@5	0.00	0.71	0.73	0.13	BM25,Dense	struct	SA-RRF	BM25
StatCan	Stat.	5.9K	553	R@5	0.18	0.00	0.18	0.56	BM25,Dense	std	SA-RRF	Dense
WatsonxD	Docs	1.1K	30	R@5	0.44	0.16	0.82	0.64	BM25,Dense	std	Rerank	BM25

ρ_{link} : link coverage; ρ_{header} : multi-level-header fraction; ρ_{len} : fraction fitting the reranker window. Graph is a candidate signal when cell links are available.

and reranking stages and reports top-1/top-5 behavior. RQ3 fixes the reader, prompt, and table budget, varying only the retriever. RQ4 uses a ReAct-style scaffold with a local Qwen3.6-35B-MoE vLLM endpoint and verifies that the same `retrieve_tables(q,k)` interface calls a live StructIR skill; its second part merges four corpora (OTT-QA+NQTABLES wiki, AIT-QA+MULTIHIERTT financial) into one 190K-table lake with source-qualified gold ids and compares fixed retrieval tools, agent-held tools, and the skill’s self-routing under one fixed reader, on every eligible query of all four corpora. A frontier-model check (DeepSeek-v4-flash, an online model deliberately stronger than and disjoint from the offline 35B used elsewhere) replicates the agent conditions on a deterministic 150-query-per-source sample.

5.1 Setup

Datasets and metrics. We evaluate on the full IBM Table-Text IR suite [IBM Research \(2025\)](#): four Wikipedia-style datasets (OTT-QA, NQTABLES, OPENWIKITABLES, FETAQA), two financial datasets (MULTIHIERTT, AIT-QA), one statistical dataset (STATCAN), and one technical-documentation dataset (WATSONXDOCS). Following the suite protocol, we report R@5 on six datasets and M@5 on MULTIHIERTT and AIT-QA. In the pipeline analysis, each cell reports top-1/top-5.

Profile-driven configuration. Table 4 is the operational output of fit: each row shows the corpus statistics observed for a corpus and the frozen retrieval plan derived from them. The plan is determined by retrieval affordances rather than dataset names: high ρ_{link} makes graph evidence available, high ρ_{header} triggers structure-aware serialization, high ρ_{idf} favors BM25-heavy fusion, and low ρ_{len} discourages reranking. The next two RQs test whether these frozen plans improve retrieval quality and explain stage-level behavior.

Baselines. We compare against three groups of baselines. First, we include basic retrieval signals and fixed fusion: BM25 [Robertson and Zaragoza \(2009\)](#), Dense retrieval with bge-m3 [Chen et al. \(2024\)](#), and Equal-RRF over BM25 and Dense [Cormack et al. \(2009\)](#). Second, we include strong pre-trained embedding models: arctic-embed-m-v2 [Yu et al. \(2024\)](#) and Granite-R2 [Awasthy et al. \(2025\)](#). Third, we include retrieval baselines: SPLADE [Formal et al. \(2021\)](#), a learned sparse first-stage

Table 5: Main retrieval results (R@5; M@5 for MultiHT and AIT-QA).

Method	OTT-QA	NQ-Tab.	OpenWiki	FeTaQA	MultiHT	AIT-QA	StatCan	WatsonxD
BM25	79.36	31.30	70.88	70.54	69.05	27.25	2.99	73.33
bge-m3 dense	93.18	76.80	97.12	93.16	53.55	39.99	24.48	66.67
Equal-RRF	89.97	59.33	94.05	92.26	65.12	43.74	19.92	66.67
arctic-embed-m-v2	93.63	80.83	97.79	93.41	58.99	46.72	26.95	70.00
Granite-R2-47M	94.44	72.35	96.13	91.56	46.63	39.42	24.85	66.67
Granite-R2-149M	94.22	79.85	97.31	92.36	49.68	42.24	28.73	66.67
SPLADE	92.68	70.48	96.21	94.86	54.65	13.71	23.27	66.67
CRAFT	93.86	79.01	97.33	93.11	50.97	27.80	29.51	66.67
StructIR	94.53	87.63	97.82	95.46	71.42	48.27	24.16	80.00

ranker, and CRAFT [Singh et al. \(2025\)](#), a training-free cascaded retriever for tabular QA.

StructIR configuration. StructIR executes the retrieval plan frozen during fit. Each online call uses the same `retrieve(q,k)` interface and executes the configured signal registry, SA-RRF weights, serialization mode, and output stage.

Reader and agent setting. For downstream QA, we use a fixed 35B reader with the same prompt, decoding parameters, and retrieved-table budget across retrievers. For agent-tool integration, we use a ReAct-style scaffold served by a local Qwen3.6-35B-MoE vLLM endpoint. The agent receives a single tool schema, `retrieve_tables(q,k)`, exposed by StructIR through `as_tool()`.

5.2 RQ1: Overall Retrieval Effectiveness

We compare StructIR with sparse, dense, fixed-fusion, pre-trained embedding, and cascaded retrieval baselines on all eight datasets (Table 5). StructIR achieves the best score on seven of eight datasets, with the largest gains on corpora where a fixed cascade applies the wrong final stage.

Fixed cascades fail when retrieval quality is stage-non-monotonic. CRAFT is competitive on some compact corpora but breaks on financial tables. On MULTI-HIERTT, StructIR reaches 71.42 while CRAFT reaches 50.97, a gain of 20.45 points; on AIT-QA, StructIR reaches 48.27 while CRAFT reaches 27.80, a gain of 20.47 points. Table 4 explains the gap: both financial corpora have high header complexity ($\rho_{\text{header}} = .58/.71$) and low reranker-window fit ($\rho_{\text{len}} = .36/.13$), so pushing candidates into a fixed reranking cascade is brittle. StructIR freezes the stopping point at fit time and stops at SA-RRF for these corpora. This validates output-stage selection as a core retrieval-skill decision.

Corpus-conditioned fusion prevents signal dilution. The largest gains over fixed fusion occur where sparse and dense signals disagree. On NQTABLES, StructIR improves over Equal-RRF by 28.30 points; on MULTIHIERTT, it improves over Equal-RRF by 6.30 points. Equal-RRF assigns the same BM25–Dense weights to every corpus, so the weaker leg can dilute the useful signal. StructIR uses corpus-conditioned

Table 6: Pipeline decomposition, reported as R@1/R@5 (M@1/M@5 for MultiHT and AIT-QA).

Stage	OTT-QA	NQ-Tab.	OpenWiki	FeTaQA	MultiHT	AIT-QA	StatCan	Watson
BM25	62.6/79.3	18.0/31.3	46.0/70.9	49.0/70.5	45.6/69.2	7.4/27.3	0.6/2.9	53.3/70.9
dense	79.4/92.9	45.3/76.8	73.8/97.1	77.8/93.2	34.5/53.6	19.1/40.0	8.9/24.5	63.3/69.2
SA-RRF	79.8/94.0	43.8/76.4	75.0/97.5	75.7/94.2	45.7/71.4	19.4/44.0	10.3/24.2	56.7/69.2
Reranker	83.1/94.5	60.7/87.6	81.6/97.8	80.7/95.5	32.4/55.6	19.2/40.3	7.0/24.1	63.3/69.2
StructIR	83.1/94.5	60.7/87.6	81.6/97.8	80.7/95.5	45.7/71.4	19.4/44.0	10.3/24.2	63.3/69.2

SA-RRF weights: dense-heavy for Wikipedia-style corpora and BM25-heavy for financial corpora with high ρ_{idf} . This validates fit-time fusion configuration as the second source of robustness.

Overall, RQ1 shows that StructIR’s retrieval gains come from configuring both fusion and stopping behavior from corpus profiles.

5.3 RQ2: Stage Non-Monotonicity

We decompose the retrieval pipeline into BM25, Dense, SA-RRF, and reranking stages (Table 6). The table tests the cascade monotonicity assumption: a later stage can improve precision on some corpora and degrade it on others.

Reranking is a precision amplifier when the corpus fits the cross-encoder view. On NQTABLES, reranking raises top-1 performance from 45.3 with Dense to 60.7, and top-5 performance from 76.8 to 87.6. On OPENWIKITABLES and FETAQA, reranking reaches top-1 scores of 81.6 and 80.7. These corpora have high ρ_{len} and standard layouts, so the cross-encoder can inspect the relevant table evidence within its input window. This validates reranking as a high-precision final stage for compact Wikipedia-style corpora.

Reranking becomes destructive on long hierarchical financial tables. On MULTIHIERTTT, SA-RRF reaches 45.7/71.4 at top-1/top-5, while reranking drops to 32.4/55.6. On AIT-QA, SA-RRF reaches 19.4/44.0 and reranking reaches 19.2/40.3. Table 4 explains this reversal: financial corpora have high ρ_{header} but low ρ_{len} , so the reranker sees a truncated or flattened view of the table, while SA-RRF preserves the BM25-heavy signal mix. This validates the stopping decision produced during fit: later stages are used only when the corpus profile supports them.

Overall, RQ2 shows that StructIR’s gains come from breaking the fixed-cascade monotonicity assumption.

5.4 RQ3: Retrieval Skill Utility under a Fixed Reader

We use downstream QA as a fixed-reader utility check (Table 7). The reader, prompt, decoding parameters, and top-1 table budget are fixed; only the retriever changes.

Better table evidence improves fixed-reader QA. StructIR gives the strongest non-oracle QA results on all four Wikipedia-style datasets. On NQTABLES, StructIR

Table 7: Fixed-reader downstream QA. EM for OTT-QA/NQ-Tab./OpenWiki; F1 for FeTaQA.

Retriever	OTT-QA	NQ-Tab.	OpenWiki	FeTaQA
BM25	10.25	17.15	34.64	23.09
dense	12.33	42.92	50.85	29.29
SA-RRF	12.29	42.83	52.44	28.52
StructIR	13.01	50.14	56.36	29.83
<i>gold table</i>	<i>14.81</i>	<i>62.51</i>	<i>61.97</i>	<i>31.86</i>

Table 8: Live agent-tool integration with StructIR. Gates compare live retrieval with committed rankings.

Dataset	<i>n</i>	Hit@5	EM	F1	Tool success	Calls/q	Top-1 gate	Top-5 gate
NQ-Tab.	150	86.67	40.67	51.92	1.00	1.233	30/30	30/30
OTT-QA	150	70.00	46.67	51.03	0.96	1.740	30/30	29/30

reaches 50.14 EM, compared with 42.92 for Dense and 42.83 for SA-RRF. On OPENWIKITABLES, StructIR reaches 56.36 EM, compared with 50.85 for Dense and 52.44 for SA-RRF. Relative to BM25 and the gold-table oracle, StructIR closes 72.7% of the evidence gap on NQTABLES and 79.5% on OPENWIKITABLES. Since the reader and prompt are fixed, these gains come from presenting better table evidence to the same model. This validates StructIR as a retrieval skill whose benefits survive downstream reading.

Overall, RQ3 shows that StructIR improves the evidence available to a fixed reader, which is the main downstream role of a table-retrieval skill.

5.5 RQ4: Agent-Tool Integration

RQ4 evaluates StructIR as a live retrieval skill inside a ReAct-style data-agent scaffold (Table 8). The agent calls the same `retrieve_tables(q,k)` interface for all queries, while StructIR executes the frozen plan inside the fitted skill instance.

StructIR provides usable table evidence through a live agent tool interface.

On NQTABLES, the live agent setting achieves 86.67 Hit@5, 40.67 EM, and 51.92 F1; on OTT-QA, it achieves 70.00 Hit@5, 46.67 EM, and 51.03 F1. The agent makes more than one call on average, with 1.233 calls per query on NQTABLES and 1.740 on OTT-QA, showing that the retrieval skill is used inside an actual tool-calling loop rather than an offline evaluator. The live retrieval also passes the committed ranking gates: 30/30 top-1 checks on both datasets, and 30/30 and 29/30 top-5 checks on NQTABLES and OTT-QA. This validates both sides of the skill claim: StructIR can be called by an agent, and the live tool returns effective table evidence.

5.5.1 The Mixed-Lake Agent Benchmark

Callability alone does not test the paper’s central claim, that per-corpus configuration is what an agent’s retrieval tool is missing. We therefore merge four corpora into one 190K-table lake (two source families; the wiki pair is content-overlapping, the

Table 9: Mixed-lake retrieval, full evaluation (190K tables, 4 sources / 2 families; R@5 / top-1 acc; gold ids source-qualified).

Condition	Routing	OTT-QA	NQ-Tab.	AIT-QA	MultiHT	micro R@5
BM25 (fixed, flat)	—	71.2/53.3	30.5/17.2	33.2/19.6	69.4/49.0	55.8
Dense (fixed, flat)	—	80.1/62.0	76.2/44.9	39.4/35.3	53.0/37.6	68.1
Cascade (fixed, rerank-always)	—	86.3/70.7	87.2/61.1	40.0/35.3	55.0/35.0	73.9
Agent + Dense tool	agent	78.6/60.3	76.7/44.8	40.6/35.3	51.6/35.7	67.5
Agent + StructIR tool	agent	79.2/65.4	87.0/60.2	39.0/32.2	64.6/41.8	73.0
StructIR (auto)	fit-time self-diagnostic	87.3/70.8	87.2/60.3	42.6/35.5	68.4/47.3	77.6
<i>StructIR (oracle family)</i>	<i>oracle</i>	<i>87.3/70.8</i>	<i>87.3/60.4</i>	<i>43.8/36.1</i>	<i>70.6/48.8</i>	<i>78.3</i>
<i>StructIR (oracle source)</i>	<i>oracle</i>	<i>93.3/80.1</i>	<i>87.6/60.8</i>	<i>43.9/36.1</i>	<i>71.4/49.5</i>	<i>81.0</i>

Table 10: Downstream QA over the lake, full evaluation: fixed per-dataset readers, EM, identical reader and prompts across conditions.

Condition	OTT-QA EM	NQ-Tab. EM
Agent + Dense tool	43.7	52.1
Dense (fixed, flat)	46.2	51.3
Agent + StructIR	47.6	51.4
Cascade (fixed)	50.8	52.1
StructIR (auto)	52.1	51.8
<i>StructIR (oracle source)</i>	<i>60.0</i>	<i>51.9</i>

Table 11: Agent-contract ablation: identical prompts and call budget; (a) episode-evidence rule, (b) union scale.

<i>(a) Episode evidence (qwen3.6-35b, agent+Dense)</i>			
	rewritten-final hit@5	verbatim-final hit@5	
Last call (naive)	48.9	84.6	
Session union	76.7	84.6	
<i>(b) Union scale (agent+StructIR, 150-query sample; micro R@5 / OTT-QA EM)</i>			
	qwen3.6-35b	v4-flash	
Raw-score union	68.0 / 50.7	71.4 / 52.7	
Per-call-rank union	65.0 / 40.7	70.0 / —	
Stage-aware union	68.4 / 53.3	71.0 / 55.3	

financial pair content-disjoint) and compare three contracts under one fixed reader: *fixed tools* (flat BM25, flat dense, and an always-rerank cascade using our own trained reranker), *agent-held tools* (an identical ReAct scaffold holding either the fixed dense tool or StructIR), and *the skill’s own routing (auto)*, with oracle-routing rows as upper bounds. Before any condition ran, two gates passed: live fit reproduces all frozen plans of Table 4 on the real structural corpora, and live per-source retrieval inside the lake reproduces the committed single-corpus scores (*e.g.*, MULTIHIERTT M@5 71.42 vs. 71.42).

Fixed tools fail in both directions (Table 9). BM25 collapses on NQTABLES (30.5); dense collapses on MULTIHIERTT (53.0, below BM25’s 69.4); the strongest fixed pipeline, the always-rerank cascade, lifts the wiki corpora but stays 14 points below the BM25 floor on MULTIHIERTT (55.0 vs. 69.4): the stage that buys its wiki gains is exactly the stage that destroys its financial slice.

The skill recovers both sides at once. StructIR’s auto row reaches micro R@5 **77.6**, +3.7 over the best fixed tool, with fit-time routing at 99.0%: the leak self-diagnostic (Section 4.3) merges the content-overlapping wiki pair and votes members on the content-disjoint financial pair. On the oracle-source row the per-source quality inside the lake matches the committed single-corpus scores (*e.g.*, MULTIHIERTT M@5 71.4); the oracle rows bound the residue at +0.7 for perfect family routing and +3.4 for perfect source identification, the latter being exactly the within-family ambiguity the diagnostic declared unrecoverable.

Same agent, swap the tool. Under the identical scaffold, prompts, call budget, and union rule, replacing the fixed dense tool with StructIR is worth +5.5 micro R@5 (73.0 vs. 67.5; +7.6 in the frontier-model sample), with the largest gains where configuration matters most (MULTIHIERTT +13.0, NQTABLES +10.3). The skill’s own routing (77.6) clearly beats the best agent-held configuration: the recommended integration passes `source="auto"` or a family hint.

Tool-argument granularity is a model policy, and the contract must be robust to both. The two models use the same tool in opposite ways. The 35B names a *concrete member* in 94% of calls: it never names the opaquely-titled financial corpus at all (routing it to the recognizably-named sibling instead), is barely above chance inside the content-overlapping wiki family, and spontaneously hedges both wiki siblings in 37% of episodes, an expensive self-invented approximation of the family merge. v4-flash instead passes a *family* argument in ~93% of calls, never hedges, and reaches 98–100% wiki routing, discovering by itself the abstraction level the self-diagnostic also selects. Both policies land within 2.6 micro R@5 because the contract absorbs the difference: member guesses are rescued by the union and the comparable-scale merge, and family hints take the optimal path directly. This shows, at two model scales, that fine-grained routing does not belong in the agent’s hands.

Contract ablation. Table 11: with last-call evidence the reformulation pathology reproduces (rewritten-final episodes 48.9 vs. 84.6 verbatim); the session union closes it to 76.7 with identical prompting. The union’s aggregation scale matters in turn (panel b): raw-score union breaks when the `source` argument changes the output stage across calls (reranker logits pollute RRF-mass evidence), pure rank union discards genuine same-scorer comparability and ties sibling near-duplicates (OTT-QA QA 50.7→40.7 on the 35B); the stage-aware rule (score within one output stage, per-call rank across stages) is best or tied on end-task QA for both models. All three rules are enforced by the harness with identical prompting.

End-task transfer. Table 10: on OTT-QA, whose answer protocol requires the source-native table, EM follows retrieval: agent+dense 43.7 < dense 46.2 < agent+StructIR 47.6 < cascade 50.8 < StructIR(auto) **52.1** ≪ oracle-source 60.0. The mechanism is sharp: conditional on the source-native table at rank 1, EM is uniform across conditions and readers; failing that, EM is exactly zero. Lake QA is gated by *evidence compatibility*, and the QA ladder is a routing ladder. RQ4’s QA is therefore diagnostic rather than a primary ranking metric; NQTABLES sits in a top-5 saturation

band (51–52 EM while hit@5 spans 77–87), a useful negative for budgeting retrieval quality against reader context.

Overall, RQ4 shows that StructIR’s profile-driven retrieval plan can be exposed as a stable agent-callable skill with measurable downstream utility, and that on a heterogeneous lake the self-configuring contract is what closes the gap that neither a better fixed pipeline nor a smarter agent policy closes.

6 Related Work

Table retrieval and table-text question answering. Table retrieval has been studied from several representation perspectives. Early work ranks web tables using semantic features over table fields, entities, captions, and surrounding metadata [Zhang and Balog \(2018\)](#); [Shraga et al. \(2020\)](#). Open-domain table QA then introduced dense retrieval over linearized tables and table-text evidence, where the retriever selects tables or passages before a reader produces the final answer [Herzig et al. \(2021\)](#); [Huang et al. \(2022\)](#); [Chen et al. \(2021a\)](#). Recent work has expanded this line with structure-aware dense retrieval, denoising table-text retrieval, field-aware hybrid matching, table-specific benchmarks, and cascaded training-free retrievers [Jin et al. \(2023\)](#); [Wang et al. \(2022\)](#); [Kang et al. \(2024\)](#); [Ji et al. \(2024\)](#); [Li et al. \(2025\)](#); [Singh et al. \(2025\)](#). These methods mainly improve how a query and a table are matched. StructIR addresses the configuration layer above individual matchers: it profiles each corpus, registers the available sparse, dense, graph, and reranking signals, and freezes a retrieval plan before online queries arrive.

Data discovery in heterogeneous data lakes. Data lakes aggregate tables from heterogeneous sources, which creates discovery problems around schema matching, unionability, joinability, metadata, and cross-modal evidence [Nargesian et al. \(2023\)](#); [Fan et al. \(2023\)](#); [Eltabakh et al. \(2023\)](#); [Wang et al. \(2025\)](#). Data discovery systems such as Aurum profile and index large collections to support discovery over schema- and instance-level relationships [Fernandez et al. \(2018\)](#). Table union search systems such as Pylon and SANTOS further show that semantic relationships between columns and entities provide useful signals beyond surface schema similarity [Cong et al. \(2023\)](#); [Khawiwada et al. \(2023\)](#). StructIR uses this data-lake view for table retrieval: corpus statistics such as link coverage, header hierarchy, lexical specificity, and reranker-window fit become the basis for configuring the retrieval skill.

Agent-callable data skills and tool interfaces. LLM data agents increasingly interact with databases through external tools and specialized data services [Fernandez et al. \(2023\)](#); [Luo et al. \(2026\)](#); [Biswal et al. \(2025\)](#). General agent work has established the usefulness of tool interfaces and interleaved reasoning/action loops, including API-using language models, ReAct-style prompting, and multi-agent platforms [Schick et al. \(2023\)](#); [Yao et al. \(2023\)](#); [Gao et al. \(2024\)](#). Recent work on agentic skills and database-learning toolkits further frames reusable data capabilities as callable skills with stable interfaces [Park et al. \(2026\)](#); [Zhang et al. \(2026\)](#). StructIR follows this recipe at the retrieval layer. The skill is configured through fit, invoked through retrieve, and exposed

to an agent through `as_tool`, while the agent’s planning policy and downstream reader remain replaceable.

Agent–tool reliability: routing, rewriting, and evidence aggregation. A growing body of work locates agent failures at the agent–tool interface rather than in the agent’s reasoning: tool selection is highly sensitive to the quality of function definitions and descriptions Guo et al. (2026), large tool inventories require retrieval and filtering layers of their own Lumer et al. (2025b), and multi-turn tool use needs explicit memory management of returned results Lumer et al. (2025a). Within data agents, recent table-discovery and lake systems give the agent a single retrieval capability with a fixed configuration Cui et al. (2025); Li and Galhotra (2026); An et al. (2025); Salemi et al. (2025). Our measurements add the retrieval-tool counterpart of these findings: agents route by the *semantics of source names* rather than source content, rewrite exactly the long multi-hop questions whose entities matter most, and cannot distinguish content-overlapping sibling corpora at any model scale. Each failure is closed by tool-side configuration (fit-time self-diagnostics) or by the harness-enforced contract (verbatim-first querying, session-evidence union).

7 Conclusion

We presented StructIR, a **corpus-profile-driven** table retrieval skill for LLM data agents. Motivated by a cross-source signal analysis showing that the relative effectiveness of dense, sparse, and graph-based signals is source-dependent, **and that the agent itself cannot compensate at query time**, StructIR profiles each corpus offline and freezes a retrieval plan (signal registry, **corpus-adaptive** reciprocal rank fusion, **serialization mode**, and **output stage**), exposing a single `fit/retrieve/as_tool` interface to the agent. **At the lake level**, `fit` additionally measures, without labels, when its own routing probe cannot distinguish sibling corpora and freezes a routing policy (member voting where the probe is reliable, comparable-scale merging where it is not), and a harness-enforced agent contract (verbatim-first querying, session-evidence union) makes autonomous tool calls monotonically helpful. Across eight table–text retrieval datasets, the frozen plans are best against nine baselines on seven of eight datasets; on a 190K-table mixed lake queried end-to-end by a tool-calling agent, every fixed retrieval tool fails on some source family while StructIR’s self-routed retrieval beats the strongest fixed pipeline with 99% routing accuracy, and the gains transfer to downstream QA under a fixed reader, consistently across a 35B local model and a frontier online model. Future work includes extending the corpus profiler to additional structural features, **scorer-identity-aware merging for lakes whose sources carry different rerankers**, and automating the onboarding of new source families as data lakes evolve.

References

An Q, Ying C, Zhu Y, et al (2025) Ledd: Large language model-empowered data discovery in data lakes. arXiv preprint arXiv:250215182

- Awasthy P, Trivedi A, Li Y, et al (2025) Granite embedding R2 models. [arXiv:2508.21085](#)
- Biswal A, Patel L, Jha S, et al (2025) Text2SQL is not enough: Unifying AI and databases with TAG. In: CIDR
- Cafarella MJ, Halevy A, Wang DZ, et al (2008) WebTables: Exploring the power of tables on the web. PVLDB 1(1):538–549. <https://doi.org/10.14778/1453856.1453916>
- Chen J, Xiao S, Zhang P, et al (2024) M3-embedding: Multi-linguality, multi-functionality, multi-granularity text embeddings through self-knowledge distillation. In: Findings of the Association for Computational Linguistics: ACL 2024, pp 2318–2335, <https://doi.org/10.18653/v1/2024.findings-acl.137>
- Chen W, Chang MW, Schlinger E, et al (2021a) Open question answering over tables and text. In: ICLR
- Chen Z, Chen W, Smiley C, et al (2021b) FinQA: A dataset of numerical reasoning over financial data. In: Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing, pp 3697–3711, <https://doi.org/10.18653/v1/2021.emnlp-main.300>
- Chung Y, Kakkar GT, Gan Y, et al (2025) Is long context all you need? leveraging LLM’s extended context for NL2SQL. PVLDB 18(8):2735–2747. <https://doi.org/10.14778/3742728.3742761>
- Cong T, Nargesian F, Jagadish HV (2023) Pylon: Semantic table union search in data lakes. [arXiv:2301.04901](#)
- Cormack GV, Clarke CLA, Buettcher S (2009) Reciprocal rank fusion outperforms condorcet and individual rank learning methods. In: SIGIR, pp 758–759
- Cui L, Jiang G, Li H, et al (2025) Tablecopilot: A table assistant empowered by natural language conditional table discovery. In: Proceedings of the VLDB Endowment (VLDB)
- Eltabakh MY, Kunjir M, Elmagarmid AK, et al (2023) Cross modal data discovery over structured and unstructured data lakes. PVLDB 16(11):3377–3390. <https://doi.org/10.14778/3611479.3611533>
- Fan G, Wang J, Li Y, et al (2023) Table discovery in data lakes: State-of-the-art and future directions. In: Companion of the 2023 International Conference on Management of Data (SIGMOD), pp 69–75
- Fernandez RC, Abedjan Z, Koko F, et al (2018) Aurum: A data discovery system. In: 2018 IEEE 34th International Conference on Data Engineering (ICDE), pp

1001–1012, <https://doi.org/10.1109/ICDE.2018.00094>

Fernandez RC, Elmore AJ, Franklin MJ, et al (2023) How large language models will disrupt data management. *PVLDB* 16(11):3302–3309. <https://doi.org/10.14778/3611479.3611527>

Formal T, Piwowarski B, Clinchant S (2021) SPLADE: Sparse lexical and expansion model for first stage ranking. [arXiv:2107.05720](https://arxiv.org/abs/2107.05720)

Gao D, Li Z, Pan X, et al (2024) AgentScope: A flexible yet robust multi-agent platform. [arXiv:2402.14034](https://arxiv.org/abs/2402.14034)

Gomm D, Hulsebos M (2025) Metadata matters in dense table retrieval. In: *ELLIS Workshop on Representation Learning and Generative Models for Structured Data*

Guo R, Dong K, Gao X, et al (2026) Learning to rewrite tool descriptions for reliable llm-agent tool use. *arXiv preprint arXiv:260220426*

Herzig J, Müller T, Krichene S, et al (2021) Open domain question answering over tables via dense retrieval. In: *NAACL*, pp 512–519, <https://doi.org/10.18653/v1/2021.naacl-main.43>

Hu Z, Dong Y, Wang K, et al (2020) Heterogeneous graph transformer. In: *WWW*, pp 2704–2710

Huang J, Zhong W, Liu Q, et al (2022) Mixed-modality representation learning and pre-training for joint table-and-text retrieval in openQA. In: *Findings of EMNLP*, pp 4117–4129, <https://doi.org/10.18653/v1/2022.findings-emnlp.303>

Hulsebos M, Demiralp Ç, Groth P (2023) GitTables: A large-scale corpus of relational tables. *Proceedings of the ACM on Management of Data* 1(1):1–17. <https://doi.org/10.1145/3588710>

IBM Research (2025) Table + text IR evaluation. <https://huggingface.co/collections/ibm-research/table-text-ir-evaluation>, hugging Face collection; accessed 2026-06-06

Ji X, Parameswaran AG, Hulsebos M (2024) TARGET: Benchmarking table retrieval for generative tasks. In: *NeurIPS 2024 Third Table Representation Learning Workshop*

Jin N, Li D, Chen J, et al (2023) Enhancing open-domain table question answering via syntax- and structure-aware dense retrieval. In: *Proceedings of the 13th International Joint Conference on Natural Language Processing and the 3rd Conference of the Asia-Pacific Chapter of the Association for Computational Linguistics (Volume 2: Short Papers)*, pp 157–165, <https://doi.org/10.18653/v1/2023.ijcnlp-short.18>

Kang D, Jung B, Kim Y, et al (2024) Denoising table-text retrieval for open-domain question answering. In: *Proceedings of the 2024 Joint International Conference on*

- Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024), pp 4634–4640
- Khatiwada A, Fan G, Shraga R, et al (2023) SANTOS: Relationship-based semantic table union search. In: Proceedings of the 2023 International Conference on Management of Data (SIGMOD), <https://doi.org/10.1145/3588689>, arXiv:2209.13589
- Li D, Bi K, Guo J, et al (2025) THYME: Tailoring table retrieval from a field-aware hybrid matching perspective. [arXiv:2503.02251](https://arxiv.org/abs/2503.02251)
- Li WZ, Galhotra S (2026) Octopus: A lightweight entity-aware system for multi-table data discovery and cell-level retrieval. arXiv preprint arXiv:260102304
- Lumer E, Gulati A, Subbiah VK, et al (2025a) Memtool: Optimizing short-term memory management for dynamic tool calling in llm agent multi-turn conversations. arXiv preprint arXiv:250721428
- Lumer E, Gulati A, Subbiah VK, et al (2025b) Scalemcp: Dynamic and auto-synchronizing model context protocol tools for llm agents. arXiv preprint arXiv:250506416
- Luo Y, Li G, Fan J, et al (2026) Data agents: Levels, state of the art, and open problems. In: SIGMOD, tutorial
- Nargesian F, Pu KQ, Ghadiri Bashardoost B, et al (2023) Data lake organization. *IEEE Trans Knowl Data Eng* 35(1):237–250
- Park J, et al (2026) SoK: Agentic skills – beyond tool use in LLM agents. arXiv preprint arXiv:260220867
- Robertson S, Zaragoza H (2009) The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval* 3(4):333–389
- Salemi A, Parmar M, Goyal P, et al (2025) Llm-based multi-agent blackboard system for information discovery in data science. arXiv preprint arXiv:251001285
- Schick T, Dwivedi-Yu J, Dessì R, et al (2023) Toolformer: Language models can teach themselves to use tools. In: *Advances in Neural Information Processing Systems*
- Shraga R, Roitman H, Feigenblat G, et al (2020) Web table retrieval using multimodal deep learning. In: *SIGIR*, pp 1399–1408
- Singh A, Bhandari KR, Gao J, et al (2025) CRAFT: Training-free cascaded retrieval for tabular QA. Accepted to ACL 2026 Main, [arXiv:2505.14984](https://arxiv.org/abs/2505.14984)
- Wang J, Feng Y, Shen C, et al (2025) Towards operationalizing heterogeneous data discovery. [arXiv:2504.02059](https://arxiv.org/abs/2504.02059)

- Wang Y, Gan J, Gao S, et al (2026) Beyond linearization: Attributed table graphs for table reasoning. arXiv preprint arXiv:260108444
- Wang Z, Jiang Z, Nyberg E, et al (2022) Table retrieval may not necessitate table-specific model design. [arXiv:2205.09843](#)
- Yao S, Zhao J, Yu D, et al (2023) ReAct: Synergizing reasoning and acting in language models. In: International Conference on Learning Representations (ICLR)
- Yu P, Merrick L, Nuti G, et al (2024) Arctic-embed 2.0: Multilingual retrieval without compromise. [arXiv:2412.04506](#)
- Zhang S, Balog K (2018) Ad hoc table retrieval using semantic similarity. In: WWW, pp 1553–1562
- Zhang Y, Xu L, Gan Q, et al (2026) RDBLearn: Simple in-context prediction over relational databases. [arXiv:2602.18495](#)